La documentazione di rsedit

Regole, inventario e piano del riordino — passo 0

Indice

1 In breve 2
2 Le regole 3
2.1 Chi legge che cosa 3
2.2 Dove sta ogni informazione 3
2.3 I manpage 4
2.4 Le docstring 5
2.5 Il rustdoc 6
2.6 I commenti nel codice 6
2.7 Il manuale per sviluppatori 6
2.8 Il rapporto sui problemi 7
2.9 La storia e i commit 7
2.10 I test 7
2.11 Le copie controllate 7
2.12 La lista di controllo di ogni patch 8
2.13 Che cosa si fa dei commenti di oggi 8
2.14 Tre esempi 8
3 L'inventario 11
3.1 Quanto c'è 11
3.2 I commenti del Rust 11
3.3 Docstring e rustdoc 12
3.4 I moduli Lisp 12
3.5 I manpage 13
3.6 I documenti in `doc/` e nella radice 13
3.7 Lo stesso fatto in più posti 14
3.8 Nomi citati che non esistono 15
3.9 Difetti trovati durante l'inventario 15
4 Il piano 16
4.1 Fase A — le regole e i controlli (passo 1, fatto) 16
4.2 Fase B — i manpage e le docstring (passi 2–9) 16
4.3 Fase C — il codice e i commenti (passi 10–21) 26
4.4 Fase D — il manuale (passo 22) 27
4.5 Come si consegna ogni passo 27
5 Le decisioni 28

1 In breve

Oggi la stessa informazione su rsedit è scritta in più posti: nei commenti del codice, nelle docstring che `C-h f` mostra, nei manpage di `man/` e nei documenti typst di `doc/`. Le copie non si aggiornano insieme e divergono. L'elenco dei moduli caricati all'avvio, per esempio, compare in sei posti, in cinque versioni diverse (§ 3.7). Nei commenti ci sono 145 sezioni `# Why` e circa 80 righe che raccontano com'era il codice prima. Il manuale per sviluppatori descrive ancora lo scheduler che le patch 1–9 hanno sostituito.

Questo documento stabilisce dove sta ogni informazione, come si scrive e come si controlla che non diverga. Poi misura lo stato attuale e propone i passi per arrivarci.

> **Stato: approvato; i passi 1–5 sono fatti.** Le misure dell'inventario si riferiscono alla revisione `58d0b0d`. Le decisioni prese sono raccolte nel § 5.

> **Le regole, in sei righe.**
> 1. **Ogni fatto ha una casa sola.** Altrove si rimanda, non si copia.
> 2. **Chi usa l'editor legge i manpage**, completi e con esempi pratici. La docstring è il promemoria breve che `C-h f` mostra, e porta alla pagina.
> 3. **Chi sviluppa legge il manuale typst**: architettura, thread, lock e decisioni, con diagrammi. Il rustdoc è il contratto breve di ogni elemento.
> 4. **Un commento dice solo ciò che il codice non può dire**: un'invariante, un ordine di lock, un perché non ovvio, un TODO.
> 5. **Nel codice c'è solo il presente.** La storia sta nei commit, i difetti nel rapporto sui problemi.
> 6. **Una copia è ammessa solo se un test la confronta con la sua fonte.**

Il § 2 è la parte permanente: vale per ogni modifica futura. Il § 3 fotografa lo stato di partenza e resta, come termine di confronto, fino all'ultimo passo. Il § 4 propone i passi del riordino e si accorcia man mano che vengono fatti.

2 Le regole

2.1 Chi legge che cosa

Chi	Dove guarda	Che cosa cerca
Chi usa l'editor	i manpage: `C-h m`, `M-x manpage`	come si fa una cosa, con esempi da copiare; i tasti; i limiti
Chi configura l'editor o scrive moduli Lisp	i manpage e le docstring: `C-h f`, `C-h v`	in più, la firma esatta di una funzione e il nome di una variabile
Chi sviluppa il Rust	il manuale in `doc/`, il rustdoc, i commenti	come è fatto il codice, perché, dove intervenire
Chi indaga un difetto	`doc/rapporto-problemi.typ` `git log`	che cosa è già noto, quando un comportamento è cambiato

2.2 Dove sta ogni informazione

Informazione	Casa	Altrove
Che cosa fa una funzione, un comando o una variabile: firma, argomenti, valore	la sua docstring	la SYNOPSIS della sua pagina la elenca; `C-h f` aggiunge il rimando alla pagina
Come si usa una funzionalità: concetti, combinazioni, esempi, limiti	il manpage del suo argomento	nessuna copia nelle docstring
I tasti	i keymap dei moduli Lisp	le sezioni KEYS delle pagine, controllate da un test; `C-h b` li calcola
I moduli caricati e il loro ordine	`DEFAULT_INIT_LISP` in `editor/boot.rs`	la sezione MODULES di `rsedit.txt`, controllata; `build.rs` e i test non ne tengono una propria
L'elenco delle pagine	la directory `man/`	l'indice di `guide.txt`, controllato; i test leggono la directory
Dove l'editor tiene i suoi file: configurazione, cache, log	il codice, `paths.rs`	la pagina `configuration`, per l'utente
La struttura: moduli, tipi principali, thread, lock, flussi	il manuale per sviluppatori, `doc/architettura-editor.typ`	la documentazione di modulo: poche righe e il rimando
Perché il codice è fatto così	le decisioni del capitolo del manuale	nel codice, al più una riga e il rimando
Invarianti, ordine dei lock, `SAFETY`, il thread su cui gira una funzione	il commento o il rustdoc accanto al codice	il manuale riassume l'ordine dei lock dell'intero editor
Il contratto di un elemento Rust: che cosa fa, errori, panic	il suo rustdoc	—
La storia: che cosa c'era prima e perché è cambiato	il messaggio di commit	ciò che ne resta valido è una decisione del manuale

Informazione	Casa	Altrove
I difetti noti e il lavoro rimandato	una scheda di `rapporto-problemi.typ`	`TODO(ID)` nel codice, finché la scheda è aperta
Il comportamento atteso, caso per caso	i test: il nome dice il comportamento	—
Come si misurano le prestazioni	`PERFORMANCE.md`	—
Che cos'è rsedit, come si compila, quali documenti ci sono e per chi	`README.md`	per tutto il resto rimanda ai manpage e ai documenti

`todo.org` è la lista personale dell'autore e resta fuori da queste regole.

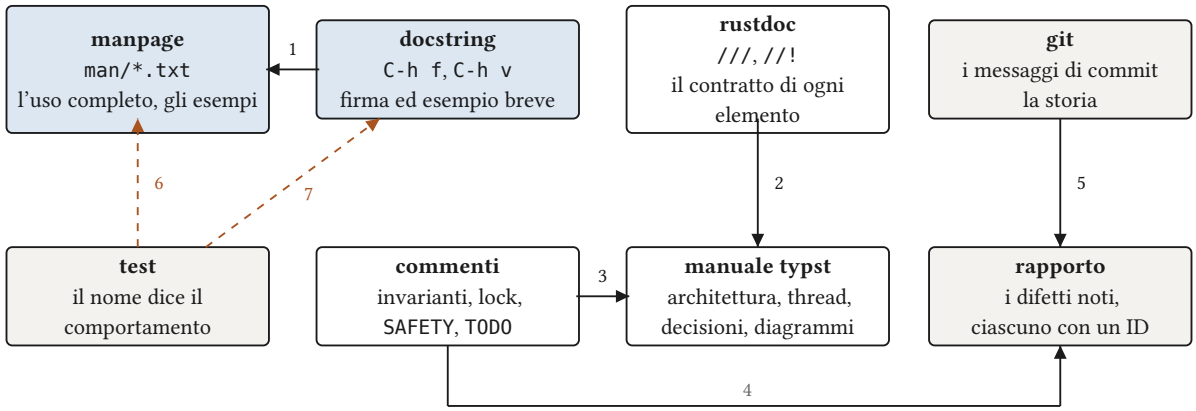


Figura 1: Le fonti e i loro rimandi. Frecce piene: «rimanda a»; tratteggiate: «controlla». In azzurro ciò che legge l'utente. (1) `C-h f` aggiunge «Manual:» con la pagina che elenca il nome nella sua SYNOPSIS. (2) La documentazione di modulo rimanda al capitolo, con un'etichetta. (3) Un perché lungo diventa una decisione del manuale. (4) `TODO(ID)` rimanda alla scheda. (5) Il commit che corregge un difetto chiude la sua scheda. (6, 7) I test confrontano le copie con le fonti e i nomi citati con quelli definiti.

2.3 I manpage

Una pagina per argomento, come oggi. Le pagine sono il riferimento completo per chi usa l'editor: ogni nome pubblico — funzione, comando, variabile, modo — ha una pagina-casa che lo elenca nella SYNOPSIS e ne spiega l'uso. Una docstring non contiene niente che manchi alla pagina.

- **SYNOPSIS** elenca i nomi di cui la pagina è la casa: `(nome ARGOMENTI)` per funzioni e comandi, il nome da solo per le variabili. Ogni nome pubblico compare nella SYNOPSIS di una sola pagina. Da qui `C-h f` e `C-h v` ricavano il rimando «Manual:», che quindi non si scrive a mano.
- **EXAMPLES** c'è in ogni pagina: esempi completi, che si possono incollare in `init.lisp` o in `M-:`, ciascuno preceduto da una frase che dice a che cosa serve.
- **KEYS** e **KEYS IN … MODE** restano come oggi: le controlla `guide_tests`.
- Le sezioni intermedie seguono ciò che il lettore vuole fare, non la struttura del codice. Il funzionamento interno va nel manuale per sviluppatori; la pagina dice ciò che l'utente vede e può fare, limiti compresi.
- In inglese, righe entro 78 colonne, intestazioni in maiuscolo a colonna 0 e corpo rientrato di quattro spazi: è ciò che `manpage-mode` riconosce e colora.

Il modello:

```
NAME
    dired - a directory in a buffer

SYNOPSIS
    (dired DIRECTORY)
    (dired-find-recursive)
    dired-sort
    find-file-recursive-limit

DESCRIPTION
    What it is for, and the idea behind it, in a few paragraphs.

OPENING A DIRECTORY
    One heading for each thing the reader wants to do.

EXAMPLES
    A sentence saying what the example is for, then the example,
    complete and ready to paste into init.lisp or M-:.

        (setq dired-sort 'type)        ; directories first

KEYS
    C-x d                   dired

KEYS IN DIRED MODE
    <ret>                   dired-find-file

SEE ALSO
    files, buffers
```

2.4 Le docstring

Valgono per le primitive (le costanti `*_DOC` di `primitives/`) e per le funzioni, i comandi e le variabili dei moduli Lisp. Seguono le convenzioni di Emacs, come già oggi.

```
(name ARGS): One sentence, in the imperative, saying what it does.

At most one short paragraph: what the arguments are, what comes back,
what is refused.

Example:
  (name ...)
```

- La prima riga basta da sola: è ciò che mostrano gli elenchi e i completamenti. Per le primitive comincia con la chiamata, `(name ARGS):`, come `doc_faithfulness_tests` già controlla.
- Gli argomenti in maiuscolo; gli altri nomi citati come `` `name' ``, la convenzione di Emacs: è così che il test dei nomi citati li riconosce.
- Un esempio quando l'uso non è ovvio.
- Di norma non più di 600 caratteri; un test rifiuta quelle oltre 1 000.
- Niente architettura, storia o motivazioni lunghe: vanno nella pagina o nel manuale. Alla pagina porta il rimando che `C-h f` aggiunge da sé.
- Le funzioni private, con `--` nel nome, hanno una riga che dice che cosa restituiscono. Un ragionamento interno è un commento `;;`, non una docstring.

- Le impostazioni che il Rust legge — variabili che nessun modulo dichiara con `defvar` — hanno la docstring nel registro `SETTINGS` di `editor/settings.rs`, che all'avvio la mette dove la mette `defvar`; il loro valore iniziale si imposta in `create_global_env`, così `C-h v` mostra anche quello. La docstring non ripete il valore iniziale.
- Un'impostazione di un modulo si dichiara con `defvar` e la sua docstring, non con un `setq` commentato: così `C-h v` la spiega e i test la vedono.

2.5 Il rustdoc

Segue le Rust API Guidelines e le convenzioni della RFC 1574.

- **La riga di riassunto** è una frase alla terza persona («Returns …», «Creates …») che basta da sola: è ciò che `cargo doc` mostra negli elenchi.
- Poi, se serve, un paragrafo breve. Per gli elementi dell'interfaccia pubblica (i `pub use` di `lib.rs`): le sezioni `# Errors`, `# Panics` e `# Safety` quando si applicano, e `# Examples`.
- I tipi e le funzioni citati sono link (`` [`Log::flush`] ``) e devono risolversi: `cargo doc --document-private-items` senza avvisi. Oggi ce ne sono 18.
- Campi e varianti: una riga quando il nome non basta, niente quando basta.
- **La documentazione di modulo** (`//!`) dice in una-cinque righe di che cosa il modulo è responsabile, e rimanda al capitolo del manuale con la sua etichetta: `Design: doc/architettura-editor.typ, <sfondo>.` Un test controlla che l'etichetta esista.
- In inglese, come oggi.

2.6 I commenti nel codice

Restano	Escono
• un'invariante su cui il codice conta e che i tipi non esprimono; • l'ordine in cui si prendono i lock, il thread su cui una funzione gira; • `SAFETY:` prima di ogni `unsafe`; • un perché non ovvio, in una o due righe, con il rimando alla decisione se ne serve di più; • il motivo di un aggiramento: un terminale, un sistema operativo; • `TODO(ID): …`, dove ID è una scheda del rapporto o un'etichetta come quelle già in uso (`uncertain`, `improve`).	• ciò che il codice dice già; • la storia: «used to», «was … until», «the bug this fixes»; • le motivazioni lunghe, le sezioni `# Why`: diventano decisioni del manuale; • le spiegazioni per l'utente: vanno nel manpage; • i riferimenti a cose che non ci sono più; • gli ID di problemi già risolti.

Nei moduli Lisp l'intestazione `;;;` dice in una-cinque righe che cosa il modulo aggiunge, e rimanda alla sua pagina e al capitolo del manuale.

2.7 Il manuale per sviluppatori

- Un solo manuale, `doc/architettura-editor.typ`, con un capitolo per sottosistema. Ogni capitolo dice di che cosa il sottosistema è responsabile e ne descrive le strutture dati principali, i thread e i lock, i flussi (con diagrammi), come si estende (le ricette) e le decisioni.
- Una decisione è una voce breve: il problema, la scelta, che cosa comporta, l'alternativa scartata e perché. Prende il posto delle sezioni `# Why` del codice.
- I diagrammi usano gli aiuti di `style.typ`. Sono obbligatori almeno per gli strati e le dipendenze dei moduli, i thread, l'ordine dei lock, il percorso da un tasto a un comando, il lavoro asincrono e la fotografia del frame.

- Il codice si cita per nome di funzione o di tipo, non per numero di riga, che invecchia a ogni modifica. Gli estratti sono brevi.
- Le etichette (`<sfondo>`) non cambiano, perché il codice vi rimanda.
- In italiano, come oggi. Il PDF si rigenera nello stesso commit della sorgente.

2.8 Il rapporto sui problemi

- Una scheda per difetto, con un ID stabile, come oggi.
- Un campo nuovo, `stato`: `aperto` oppure `risolto`, con il commit che lo risolve. Le schede risolte escono dal riepilogo e passano in un'appendice compatta.
- Il codice cita un ID solo in un `TODO(ID)`, e solo finché la scheda è aperta.

2.9 La storia e i commit

Un commento che racconta la storia si cancella. Prima si guarda se contiene una decisione ancora valida: se sì, la decisione va nel manuale. Il resto della storia sta nel messaggio di commit, che dice che cosa cambia, perché, che cosa c'era prima e quale scheda chiude.

> Per questo le patch si applicano con `git am`, che conserva il loro messaggio di commit (decisione 7 del § 5).

2.10 I test

- Il nome di un test descrive il comportamento, come già si fa: `a_variable_set_but_empty_counts_as_not_set`.
- Un commento in un test spiega in una o due righe un caso non ovvio. La storia del difetto che il test previene va nel commit.
- I test di documentazione sono quelli che controllano le copie (§ 2.11).

2.11 Le copie controllate

Copia	Fonte	Test
le sezioni KEYS	i keymap	`guide_tests` (esiste)
i SEE ALSO	la directory `man/` e le funzioni	`guide_tests` (esiste)
l'indice di `guide.txt`	la directory `man/`	`guide_tests` (esiste)
la prima riga delle docstring delle primitive	il nome della primitiva	`doc_faithfulness_tests` (esiste)
la sezione MODULES di `rsedit.txt`	`DEFAULT_INIT_LISP`	`guide_tests`
le SYNOPSIS	i nomi definiti	`guide_tests`
i nomi citati in pagine e docstring	i nomi definiti	`doc_faithfulness_tests`
i rimandi dal codice al manuale	le etichette typst	`doc_pointer_tests`

I test partono con l'elenco delle eccezioni di oggi — un nome senza pagina, una docstring troppo lunga — che può solo accorciarsi. Gli elenchi stanno in `core/src/tests/doc_exceptions/`, raggruppati per il passo che li smaltisce, e un nome che vi resta dopo essere stato sistemato fa fallire il test. Così nessun passo deve correggere tutto subito, e nessuno può peggiorare le cose.

2.12 La lista di controllo di ogni patch

- Comportamento visibile cambiato ⟶ la pagina, con i suoi esempi.
- Nome pubblico nuovo ⟶ la docstring e la SYNOPSIS della sua pagina.
- Struttura, thread o lock cambiati ⟶ il capitolo del manuale e il suo diagramma.
- Una scelta non ovvia ⟶ una voce tra le decisioni.
- Difetto corretto ⟶ la scheda passa a `risolto` e il `TODO(ID)` si toglie.
- Nessun commento storico, nessun rimando a cose tolte.
- Test verdi, compresi quelli di documentazione; `cargo doc` senza avvisi; i PDF rigenerati.

2.13 Che cosa si fa dei commenti di oggi

Ogni commento e ogni docstring esistente rientra in uno di questi casi. La tabella è lo strumento dei passi delle fasi B e C.

Il commento…	Che cosa se ne fa	Esempio, revisione `58d0b0d`
ripete il codice	si cancella	—
racconta la storia	si cancella; una decisione ancora valida va nel manuale	`managers/log.rs` : « `log_diagnostic` used to take the list's lock and, still holding it, `write_all` to the file…»
cita un problema risolto	si toglie l'ID, la regola resta	`Log::flush` : «A write that fails is dropped, never a panic (AVV-3)»
spiega all'utente come usare qualcosa	va nel manpage	`DEFAULT_INIT_LISP` rimanda a `common-keymaps.lisp` per spiegare `auto-save` e `watch-files` ; la docstring di `background-call` (§ 2.14)
motiva una scelta di progetto	va tra le decisioni del manuale; sul posto, se serve, una riga e il rimando	`paths.rs` : «Why macOS gets `~/.config` and `~/.cache` »
enuncia un'invariante, un ordine di lock, un thread	resta, in una-tre righe	`Log::flush` : «The file's lock is taken before the lines are…»
descrive lavoro da fare	una scheda del rapporto e `TODO(ID)`	`paths.rs` : il TODO sulla scrivibilità della directory
è falso, o cita cose che non esistono	si corregge o si cancella	`lib.rs` elenca un modulo `lisp/` , che è nel crate `risp` ; cinque rimandi a `rsedit-man` (§ 3.8)

2.14 Tre esempi

2.14.1 Una docstring: `background-call`

Prima del passo 8 la docstring era lunga 3 145 caratteri, 3 689 contati nel sorgente come li conta l'inventario. Eccone l'inizio:

```
(background-call NAME JOB &optional ON-PROGRESS ON-DONE INTERVAL): Do JOB in
the background under the name NAME, and return NAME at once without waiting
for it.

This is how anything that may take longer than a frame is started. JOB is
either a function of no arguments -- called once, in the background -- or a
`fiber', which is resumed a step at a time and is the shape to use when the
work can be done in pieces:
  [un esempio di nove righe e altri otto paragrafi]
```

Quasi tutto ripete la pagina `background`, che ha lo stesso esempio e le stesse regole. Dopo il riordino, che il passo 8 ha fatto:

```
(background-call NAME JOB &optional ON-PROGRESS ON-DONE INTERVAL): Run JOB in
the background under NAME, and return NAME at once. JOB is a function of no
arguments, called once, or a `fiber', resumed a step at a time as
`define-worker' resumes one. ON-PROGRESS is called with NAME after each step
but the last, and ON-DONE with NAME and t when JOB reached its end, or nil
when it failed; both run between commands. A job started under a name in use
replaces the one running, whose callbacks are then not called. INTERVAL is how
many milliseconds a plain (yield) waits: 40 unless given.

Example:
(defvar my-sum nil)
(defun my-done (name ok) (message "%s finished: %s" name ok))
(background-call 'my-sum (lambda () (setq my-sum (my-slow-sum))) nil 'my-done)

Manual: background
```

L'ultima riga non è scritta nella docstring: la aggiunge `C-h f`, dalla SYNOPSIS della pagina.

2.14.2 Un rustdoc: `app_dir`

Oggi:

```
/// The directory OPTIONS asks for, where this system keeps it -- made first,
/// when it is missing and OPTIONS says to. See the module documentation for
/// where that is.
```

Dopo, con le sezioni della RFC 1574:

```
/// Returns the directory `options.dir` names on this system, creating it and
/// its parents first when it is missing and `options.create` is set.
///
/// Where each directory is: the `configuration` manual page.
///
/// # Errors
///
/// [`DirError::Undetermined`] when nothing in the environment says where the
/// directory is; [`DirError::NotFound`] when it is missing and `create` is
/// false; [`DirError::NotADirectory`] or [`DirError::CannotCreate`] otherwise.
///
/// # Examples
///
/// ```no_run
/// use rsedit_core::{AppDir, DirOptions, app_dir};
///
/// let cache = app_dir(DirOptions { dir: AppDir::Cache, create: true })?;
/// # Ok::<(), rsedit_core::DirError>(())
/// ```
```

La tabella delle precedenze, oggi nella documentazione del modulo, è un'informazione per l'utente: passa nella pagina `configuration`, che ne ha già una parte. Le due sezioni `# Why`, sul macOS e sul Windows, diventano decisioni del capitolo sui file dell'editor.

2.14.3 Una documentazione di modulo: `managers/log.rs`

Oggi sono 29 righe:

```
//! Diagnostics: what has been logged, and where a copy of it goes.
//!
//! # Why these two are one thing
//!   [7 righe: la lista e il file si aggiornano insieme, e perché]
//! # Who writes the file
//!   [10 righe: append non scrive, flush sì, e lo chiama il watchdog]
//!
//! `log_diagnostic` used to take the list's lock and, still holding it,
//! `write_all` to the file, which put a disk write inside a lock every
//! diagnostic in the editor had to queue behind. The flush writes with the
//! list's lock given back.
```

Dopo:

```
//! Diagnostics: the list of what has been logged, and the file it is
//! copied to. [`Log::append`] only records; [`Log::flush`], which the
//! WorkManager's watchdog calls, writes what is owed.
//!
//! Design: doc/architettura-editor.typ, <log>.
```

Il primo `# Why` diventa la decisione «La lista e il file si aggiornano insieme». Chi scrive il file, e su quale thread, va nel capitolo sui thread. La storia si cancella (§ 2.9).

3 L’inventario

Misure sulla revisione `58d0b0d`, ottenute con script che contano le righe e cercano i nomi; sono fotografie, non giudizi.

3.1 Quanto c’è

Parte	codice	documentazione	commenti	quota
Rust, `core/src` e `src`, senza i test	23 832	8 433	2 699	32 %
Rust, test	34 439	2 699	3 020	14 %
Lisp, `core/lisp` (22 moduli)	3 033	1 004	1 632	46 %
manpage, `man/` (22 pagine)	—	2 225	—	—
typst, `doc/` (5 sorgenti)	—	5 865	—	—

Righe. Nel Rust «documentazione» sono le righe `///` e `//!`, «commenti» le righe `//`; nel Lisp «documentazione» sono le docstring e «commenti» le righe `;`. La quota è documentazione più commenti sul totale. Le docstring delle primitive sono stringhe Rust e qui contano come codice: 2 491 righe.

3.2 I commenti del Rust

Modulo	codice	documentazione	commenti	quota
`primitives/`	11 628	1 604	1 030	18 %
`editor/`	2 423	1 527	519	46 %
`work/`	1 996	667	72	27 %
`buffer/`	1 533	885	175	41 %
`modes/`	1 403	682	186	38 %
`managers/`	1 366	1 133	212	50 %
`ui/`	1 348	724	187	40 %
`text/`	755	491	64	42 %
`src/` (la TUI)	629	134	170	33 %
`input.rs`	241	120	8	35 %
`feature/`	211	201	56	55 %
`commands.rs`	136	104	3	44 %
`paths.rs`	132	102	7	45 %
`lib.rs`	31	59	10	69 %

La quota bassa di `primitives/` è un effetto del conteggio: le sue docstring contano come codice.

Le righe di commento fuori dai test sono 11 132. Di queste, 3 269 stanno in 128 blocchi di almeno 15 righe consecutive, e 17 blocchi superano le 40. Alcune formule segnalano i commenti da rivedere:

Formula	fuori dai test	nei test
storia: «used to», «no longer», «the bug this…»	78	57
sezioni `# Why`	145	16
«for now», «today», «not yet»	38	5
ID del rapporto (`AVV-3`, …)	2	5

Formula	fuori dai test	nei test
`TODO`	4	0

Le righe storiche sono per lo più storia vera («It used to keep the renderer waking every 40 ms»), con qualche falso positivo («text that no longer exists»). I quattro `TODO` sono in `editor/log.rs`, `paths.rs`, `primitives/ui.rs` e `src/main.rs`; nessuno rimanda a una scheda.

3.3 Docstring e rustdoc

Misura	Valore
docstring delle primitive	296 costanti, 2 491 righe di sorgente
lunghezza: mediana, 90° percentile, massimo	469, 1 089, 3 689 caratteri (`background-call`)
oltre 1 500 caratteri	8: `background-call`, `set-transient-keymap`, `define-worker`, `shell-command-start`, `directory-files-recursive` e altre tre
con un esempio; con un rimando a una pagina	230; 2
docstring dei moduli Lisp	251; mediana 161 caratteri, massimo 1 656 — una funzione privata, `electric-pair--closed-ahead-p`
definizioni Lisp private (`--`)	212 su 319
blocchi `///`	1 564: 919 di una-tre righe, 530 di quattro-dieci, 98 di 11–20, 17 oltre le 20
documentazione di modulo `//!`	103 file: 16 senza, 16 di una-cinque righe, 20 di 6–20, 50 di 21–60, 1 oltre (`primitives/lisp_job.rs`, 78)
avvisi di `cargo doc --document-private-items`	18, quasi tutti link a moduli privati

3.4 I moduli Lisp

Modulo	righe	codice	commenti	docstring	intestazione
`vim`	1 067	1 006	19	24	5
`dired`	618	221	174	142	51
`help`	565	331	72	88	34
`completion`	512	162	158	123	45
`manpage`	485	162	119	134	32
`indent`	410	161	106	85	32
`common-keymaps`	399	108	250	2	0
`electric-pair`	375	122	87	110	16
`compile`	334	155	95	43	26
`theme`	206	96	46	34	30
`buffer-list`	198	104	47	18	29
`completion-at-point`	185	44	61	55	35
`occur`	180	89	46	15	26
`risp-mode`	143	34	68	19	32

Modulo	righe	codice	commenti	docstring	intestazione
`preview`	139	50	55	19	34
`rust-mode`	131	35	71	3	12
`debug`	118	40	43	22	0
`shell`	105	45	34	9	28
`cc-mode`	95	44	24	3	1
`commands`	80	9	20	39	0
`clipboard`	47	6	31	6	22
`minibuffer`	31	9	6	11	0

Righe; «intestazione» è il blocco `;;;` in testa al file, compreso nei commenti. Le intestazioni sono saggi di progetto: quella di `dired` spiega in 51 righe perché lo stato è il buffer stesso e perché il formato è così semplice. Le docstring delle funzioni private fanno spesso da commento: quella di `dired--parent` (992 caratteri) racconta come la funzione era scritta prima.

3.5 I manpage

Le 22 pagine sono lunghe 2 225 righe. Hanno tutte NAME e DESCRIPTION; 21 hanno SEE ALSO, 7 una sezione KEYS, 3 una SYNOPSIS, nessuna una sezione EXAMPLES. Gli esempi di codice sono concentrati in tre pagine (`writing-commands`, `configuration`, `background`); dieci pagine non ne hanno.

Misura	Valore
nomi pubblici: primitive e definizioni dei moduli	487, di cui 234 citati in almeno una pagina (48 %)
comandi	180, di cui 129 citati (72 %)
completamente coperti	`overlays`, `virtual_text`, `workers`
quasi scoperti	`completion.lisp` 1 su 11, `primitives/completion.rs` 1 su 8, `help.rs` 1 su 6, `mouse.rs` 1 su 6, `minibuffer.rs` 2 su 12, `results.rs` 2 su 9; nessuno di `completion-at-point`, `preview`, `clipboard`
impostazioni lette dal Rust	18: 4 citate in una pagina, 1 sola (`help-key`) con una docstring per `C-h v`
moduli senza una pagina che li descriva	`dired` (solo dentro `files`), `shell`, `indent`, `electric-pair`, `clipboard`, `preview`, `cc-mode`, `completion-at-point`
argomenti senza pagina	il log e la diagnostica, il mouse, l'indentazione

La copertura conta un nome come coperto se una pagina lo nomina anche solo di passaggio: le pagine che lo spiegano davvero sono meno.

3.6 I documenti in `doc/` e nella radice

File	righe	Stato	Destinazione
`architettura-editor.typ`	2 642	Buono fino al lavoro asincrono. Il capitolo «Il lavoro in background» descrive lo scheduler tolto (`BackgroundScheduler`, `RunNow`, `WorkerMessage`, `background/worker.rs`); il log è ancora «nella directory corrente»; ritmi di 40 ms che non esistono più; l'appendice delle	il manuale per sviluppatori: capitoli riscritti nella fase C

File	righe	Stato	Destinazione
		primitive è una copia generata e già vecchia.	
`workmanager.typ`	786	Documento di progetto: si dichiara «da implementare», mentre le patch 1–9 l'hanno implementato; il piano e le decisioni sono storia.	nel capitolo sul lavoro asincrono, poi si toglie
`lisp-fibers.typ`	870	In inglese. Descrive l'interno del crate `risp`: tutti i suoi percorsi e 16 dei suoi identificatori sono là.	il repository `risp` (decisione 5)
`rapporto-problemi.typ`	1 244	80 schede sulla revisione `08a853d`, nessuna segnata come risolta. Le 29 delle aree LET e INT riguardano l'interprete, ora in `risp`; rimanda ad `architettura-interprete.typ`, spostato in `risp` con `10598fa`. Almeno AVV-1, AVV-3, AVV-4 e BKG-1 sembrano risolte dalle patch.	aggiornato nel passo 10
`style.typ`	323	Gli aiuti condivisi per i diagrammi.	resta
`lisp-interpreter.pdf` `lisp-interpreter-flow.pdf`	—	PDF senza sorgente, dal 24 settembre.	da togliere (decisione 4)
`README.org`	—	Dice `Ctrl+S` per salvare e `Ctrl+Q` per uscire (sono `C-x C-s` e `C-x C-c`; `C-s` cerca); ripete metà della pagina `vim`; rimanda a `M-x rsedit-man`, che non esiste.	presentazione e compilazione, poi i rimandi
`PERFORMANCE.md`	—	Valido, salvo l'ultimo paragrafo, un'istruzione di una volta sola per applicare due patch.	resta, senza l'ultimo paragrafo
`man/README.md`	—	Dice che le pagine sono per argomenti e che la docstring «is the reference»: il contrario della regola nuova.	riscritto nel passo 1

3.7 Lo stesso fatto in più posti

Fatto	Dove	Come divergono
I moduli caricati all'avvio	`DEFAULT_INIT_LISP` (22 moduli), `build.rs` (22), `shipped_lisp_tests` (19), `doc_faithfulness_tests` (18), la sezione MODULES di `rsedit.txt` (19), `architettura-editor.typ` (21)	A `shipped_lisp_tests` mancano `cc-mode`, `preview` e `completion-at-point`; a `doc_faithfulness_tests` mancano `preview`, `theme`, `vim` e `completion-at-point`, le cui docstring quindi non sono controllate. La pagina non ha `cc-mode`, `preview` e `vim`; il manuale non ha `vim`.
L'elenco delle pagine	la directory `man/` (22), l'elenco di `guide_tests` (21), l'indice di `guide.txt`	Manca `vim`: la guida non la elenca, `rsedit.txt` non la cita e i suoi tasti non sono controllati.

Fatto	Dove	Come divergono
Dove sta la configurazione	`paths.rs`, `configuration.txt`, `architettura-editor.typ`	Pagina e manuale non dicono nulla della cache e del log; il manuale ha il log nella directory corrente.
Le impostazioni	le costanti dei moduli Rust, la tabella del manuale, i commenti di `DEFAULT_INIT_LISP` e di `common-keymaps.lisp`	`configuration.txt` dice che `C-h v` spiega ogni variabile: vale per 1 impostazione su 18.
L'ordine dei moduli	`DEFAULT_INIT_LISP`, `configuration.txt`	«The order matters» contro «None of them needs any other».
Le primitive	il registro (`into.function`, `into.command`), l'appendice del manuale	L'appendice ne dà 4 a `workers` (sono 9) e 47 a `edits` (sono 48).
`background-call` e i lavori Lisp	la docstring, `background.txt`, `workmanager.typ`, il manuale, le 78 righe di `//!` di `primitives/lisp_job.rs`	Stesso esempio e stesse regole in tre posti; il manuale descrive lo scheduler di prima.
La modalità vim	`vim.txt`, `README.org`	Il README ripete comandi e impostazioni.

3.8 Nomi citati che non esistono

- `rsedit-man`, in cinque posti: `man/README.md`, `README.org`, la docstring di `data-directory`, un commento di `build.rs` e `vim.lisp`. Il comando è `manpage`.
- Nelle docstring: `read-command-args` (in `primitives/commands.rs`), `search-forward` (in quella di `scan-buffer`), `find-file-recursive` (il comando è `dired-find-recursive`), `default-directory` (in quella di `grep`). Chi le legge con `C-h f` e segue il nome non trova nulla.
- Nei commenti: `set-mode-keywords` (`rust-mode.lisp`); nomi di Emacs al posto di quelli di rsedit, come `self-insert-command`.
- Nei documenti typst: 8 identificatori del manuale e 5 di `workmanager.typ` non esistono più, come il `background/mod.rs` citato dal rapporto; 16 percorsi del rapporto e tutti quelli di `lisp-fibers.typ` sono ora nel crate `risp`.

3.9 Difetti trovati durante l'inventario

Non sono di documentazione, ma l'inventario li ha fatti emergere.

1. **`:help` di vim non fa nulla.** `vim.lisp` chiama `rsedit-man` solo se esiste, e non esiste: il comando resta muto.
2. **La pagina `vim` non si raggiunge dalla guida** e i suoi tasti non sono controllati (§ 3.7).
3. **I moduli `theme`, `vim`, `preview` e `completion-at-point` sfuggono a `doc_faithfulness_tests`**, che carica un elenco suo di moduli.
4. **`configuration.txt` consiglia `(eval-file "clipboard")`**, che l'`init.lisp` predefinito carica già.

4 Il piano

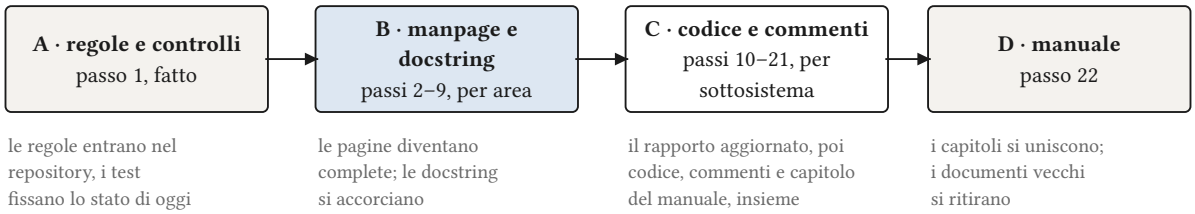


Figura 2: Le quattro fasi. Ogni passo è una patch, consegnata e verificata come le patch 1–9.

L'ordine è quello concordato: prima i manpage insieme alle docstring, poi il codice e i commenti modulo per modulo, infine il manuale. Con una precisazione: i perché tolti dal codice nella fase C vanno subito nel capitolo del manuale del loro sottosistema, così nessuna informazione resta senza casa tra un passo e l'altro. La fase D unisce i capitoli, disegna i diagrammi d'insieme e ritira i documenti vecchi (decisione 9).

4.1 Fase A — le regole e i controlli (passo 1, fatto)

1. `README.md` prende il posto di `README.org`: una presentazione essenziale del progetto, con i documenti e i loro destinatari. `man/README.md` descrive il modello di pagina del § 2.3.
2. **Un solo elenco dei moduli.** `build.rs` copia tutta `core/lisp/`; i test leggono l'elenco da `DEFAULT_INIT_LISP`; `guide_tests` confronta con quello la sezione MODULES di `rsedit.txt`, che ora ha tutti i 22 moduli.
3. **Un solo elenco delle pagine.** I test leggono la directory `man/`.
4. **I test nuovi**, con le eccezioni di oggi in `core/src/tests/doc_exceptions/`: 459 nomi senza casa, 30 docstring oltre i 1 000 caratteri, 68 funzioni private di `vim.lisp` senza docstring, 11 nomi citati che l'editor non definisce (5 variabili lette dal Rust e mai dichiarate, 6 nomi sbagliati).
5. **Il rimando automatico.** `C-h f` e `C-h v` mostrano «Manual: pagina», e `K` o `RET` su quella riga apre la pagina.
6. **Le correzioni del § 3.9.** `:help` di vim apre la pagina `vim`, che ora è nella guida, in `rsedit.txt` e nei test; tutti i moduli sono controllati. Le SYNOPSIS di `compile`, `themes` e `vim` elencano nomi, e i loro tasti sono passati in sezioni KEYS controllate.
7. Tolti i due PDF orfani e l'ultimo paragrafo di `PERFORMANCE.md`.

4.2 Fase B — i manpage e le docstring (passi 2–9)

Ogni passo copre un'area. Le pagine dell'area si riscrivono sul modello, con SYNOPSIS, EXAMPLES e KEYS; le pagine mancanti si scrivono. Le docstring dei nomi dell'area si accorciano sul modello. Le spiegazioni per l'utente che stanno nei commenti del codice dell'area passano nelle pagine e dai commenti escono. Le impostazioni dell'area ricevono una docstring, così che `C-h v` le spieghi. L'elenco dei nomi senza casa si accorcia di conseguenza.

Passo 2, fatto. `configuration` è riscritta sul modello e corretta; la pagina nuova `diagnostics` spiega l'area dei messaggi, il log, il file di log nella cache, i comandi che falliscono e le righe del watchdog. Il registro `SETTINGS` documenta le prime cinque impostazioni (`echo-message-timeout`, `mode-line-format`, `display-line-numbers`, `display-line-numbers-width`, `lisp-path`), e i passi seguenti vi aggiungono quelle della loro area. I nomi di `commands.lisp` passano al passo 8, perché la loro pagina è `writing-commands`.

Passo 3, fatto. `editing` è riscritta sul modello: muoversi, scrivere, le coppie di delimitatori, le righe, i commenti, annullare. Le pagine nuove sono due invece di una: `indentation`, come previsto, e `killing`, che prende da `editing` il kill ring, la regione e gli appunti di sistema; in una pagina sola la SYNOPSIS avrebbe superato i novanta nomi. `rectangles` e `macros` seguono il modello, con i tasti nelle sezioni KEYS. Le 107 docstring delle sei primitive dell'area e quelle dei tre moduli seguono il modello, e i ragionamenti

interni delle funzioni private sono diventati commenti `;;`. Cinque impostazioni che i moduli assegnavano con `setq` (`tab-width`, `tab-always-indent`, `electric-pair-mode`, `electric-pair-skip-self`, `clipboard-sync`) sono ora `defvar` con docstring; `clipboard-sync-toggle` è un comando. I commenti di `common-keymaps.lisp` che spiegavano all'utente i tasti dell'area sono diventati rimandi alle pagine.

Le pagine e le docstring dell'area dicevano cose false, ora corrette:

- `current-column` contava «le colonne, non i caratteri»: conta i caratteri, come `current-indentation`, e un tab vale uno; l'avvertenza in testa a `indent.lisp` era sbagliata allo stesso modo;
- `C-u 5` seguito da un carattere lo inserisce una volta sola, non cinque;
- `yank` non mette il mark all'inizio del testo inserito;
- `transpose-lines` sposta in su la riga corrente, e point con essa;
- `insert-pasted-text` restituisce sempre nil, non t;
- `set-mark` prende un secondo argomento, INCLUSIVE, che la firma taceva;
- `rectangles` diceva che un tab occupa una cella dello schermo (TUI-3 nel rapporto);
- tre C-k di fila prendono una riga e mezza, non tre righe (`first-steps`);
- l'esempio di `define-kbd-macro` legava `C-c t`, il tasto dei temi;
- i commenti dicevano il kill ring «da fare» e `risp-mode` «assente».

I tasti Canc, Inizio, Fine, PagSu e PagGiù e i tasti funzione non arrivano all'editor (CMD-2 nel rapporto): `editing` lo dice tra i limiti, e il legame di `<delete>` in `electric-pair.lisp` resta senza effetto finché la scheda è aperta.

Passo 4, fatto. `files`, `buffers` e `windows` sono riscritte sul modello, e `dired` e `mouse` sono nuove, come previsto. Il registro `SETTINGS` documenta nove impostazioni dell'area, con il loro valore iniziale: `auto-save`, `auto-save-interval`, `auto-save-directory`, `watch-files`, `watch-file-interval`, `*open-directory-callback*`, `mouse-mode`, `window-separator` e `after-resize-hook`. Le spiegazioni che stavano nei commenti di `common-keymaps.lisp` e dell'`init.lisp` predefinito sono passate in `files`. Le impostazioni di `dired.lisp` erano `defconst` e ora sono `defvar`, così un valore scelto prima che il modulo si carichi resta. Le 67 docstring delle primitive pubbliche dell'area e quelle dei due moduli seguono il modello. `all-buffer-names` trova casa in `buffers` ora invece che nel passo 6.

Le pagine e le docstring dell'area dicevano cose false, ora corrette:

- due finestre sullo stesso buffer non condividono il punto: ciascuna ricorda il suo;
- un file cambiato sul disco viene riletto quando il buffer non ha modifiche, e la barra non segnala nulla: `files` diceva il contrario su entrambi i punti;
- chiudere un buffer porta le sue finestre sul buffer usato più di recente, non su `*scratch*`;
- `delete-window` accetta un id, che la firma taceva;
- una `defun` non sostituisce una primitiva, quindi il doppio clic non si ridefinisce, come invece prometteva la docstring di `mouse-start-selection`;
- `C-g` non chiude un prompt (MIN-2 nel rapporto) né una domanda a un tasto, mentre `guide` e `first-steps` dicevano di sì;
- `first-steps` mandava a `M-x mouse-mode-toggle` per accendere il mouse, che l'`init.lisp` predefinito accende già;
- i commenti dicevano `watch-file-interval` e `auto-save-interval` letti una volta all'avvio: si leggono a ogni giro;
- i commenti dicevano che ogni mezzo minuto si copia ogni buffer modificato: ogni mezzo minuto se ne guardano quattro, a turno, e contano tutti, anche quelli senza file come `*scratch*`; con dodici buffer un file si copia ogni minuto e mezzo.

Il passo ha trovato due difetti del codice, che non corregge: li corregge la patch descritta dopo il passo 5.

- `auto-save-directory` non espande `~` e non crea la cartella, quindi il valore che l'`init.lisp` predefinito suggeriva, `"~/.cache/rsedit"`, non funzionava. Il suggerimento ora è `(expand-file-name "~/.cache/rsedit")`, e la pagina `files` dice che la cartella deve esistere.
- `BUFFERS_PER_TURN`, in `autosave.rs`, dice quanti buffer un giro può scrivere, ma conta quelli guardati: chi tiene aperti molti buffer aspetta la copia ben più di mezzo minuto. La pagina ora lo dice.

Passo 5, fatto. `searching` e `compile` sono riscritte sul modello. La pagina nuova è `regexps`, che il piano non prevedeva: la sintassi delle espressioni regolari non era scritta da nessuna parte, e le funzioni dei dati del match (`string-match` e le altre sei), `save-match-data` e `scan-buffer` vi trovano casa. Il registro `SETTINGS` documenta `case-fold-search`, `case-replace` e `*replace-read-function*`, e le ultime due hanno ora un valore all'avvio. In `compile.lisp` `compilation-patterns` passa da `setq` a `defvar` con docstring, e le altezze delle finestre, `compilation-window-height` e `occur-window-height`, da `defconst` a `defvar`. Le 41 docstring delle primitive dell'area e quelle dei due moduli seguono il modello. I commenti di `common-keymaps.lisp` sulla ricerca sono diventati rimandi, e il suo `(setq case-fold-search t)`, che il Rust fa già, non c'è più.

Le pagine e le docstring dell'area dicevano cose false, ora corrette:

- durante `M-%`, Return non si limita a fermare la sostituzione: la ferma e poi va a capo, come fa ogni tasto che non risponde alla domanda; `searching` lo dava come il tasto per fermarsi, e taceva `q`;
- `C-s` evidenzia solo la corrispondenza su cui si trova, non tutte;
- `replace-string` e `replace-regexp` non sostituiscono «dappertutto», ma dal cursore in avanti, o nella regione; senza corrispondenze restituiscono nil, non un numero;
- `string-match` distingue sempre maiuscole e minuscole, qualunque sia `case-fold-search`, e per lei `^` e `$` sono gli estremi della stringa;
- solo la ricerca nei file si riempie man mano, non quella nel buffer;
- `grep` cercava «sotto `default-directory`», che non esiste: cerca nella cartella in cui l'editor è partito; `grep--scan` rimandava a `find-file-recursive` e `scan-buffer` a `search-forward`, che non esistono;
- `scan-buffer` prometteva i gruppi `\1` e seguenti a chi legge le voci, che non li portano.

Il passo ha trovato quattro difetti del codice, che non corregge: li corregge la patch descritta qui sotto.

- dopo `C-s` e Return il messaggio dice «Mark saved where search started», ma il mark resta all'altra estremità della corrispondenza;
- `occur--display` mostra la lista nella striscia in basso e anche nella finestra in cui si lavorava, al posto del buffer; per la stessa ragione `o` non lascia vedere la corrispondenza;
- Return nell'output di una compilazione divide la striscia: il file la occupa quasi tutta, l'output resta alto una riga e il cursore passa al file; `compilation-goto` non sposta il punto da cui riparte `M-g n`;
- la prima riga dell'output, il comando, è letta come un luogo quando il comando ne contiene uno.

I difetti corretti. Una patch a parte corregge i sei difetti dei passi 4 e 5, e altri quattro trovati mentre li verificava; le pagine non li descrivono più tra i limiti, e ognuno ha un test che fallisce senza la correzione:

- dopo `C-s` e Return il mark va dove la ricerca è partita, come dice il messaggio;
- la lista di `M-s o` e `M-s g` si apre nella striscia in basso e ne prende il focus, mentre la finestra in cui si lavorava resta com'era; Return e `o` aprono la corrispondenza in quella finestra;
- Return nell'output di una compilazione apre il file nella finestra da cui la compilazione è partita, senza dividere la striscia, e `M-g n` riparte da lì;
- la riga del comando non è più letta come un luogo;
- `auto-save-directory` espande `~` e la cartella si crea alla prima copia;
- ogni giro dell'auto-save guarda tutti i buffer e scrive al più quattro copie;
- una seconda lista, o una seconda compilazione, apriva un'altra striscia sotto la prima: ora usa la stessa;

- nell'output di una compilazione `M-g n` segnava la riga sbagliata, quella che segue l'indice della voce, giusta solo per le liste di ricerca; ora una voce di `results-put` può dire, con un nono elemento, su quale riga della lista sta;
- `M-g n` su un file che non esiste apriva un buffer vuoto con quel nome: ora dice che il file manca, come già faceva Return nell'output;
- `C-h f` su una macro diceva che non è una funzione: ora la descrive, con argomenti e docstring. `defmacro` di `risp` lascia la docstring nel corpo, e la primitiva privata `help--macro-doc` la legge da lì.

`compilation-mark-priority` non c'è più: i segni sono quelli della navigazione comune, e il codice di `compile.lisp` che li metteva da sé è stato tolto con loro.

Passo 6, fatto. `completion`, `keymaps` e `getting-help` sono riscritte sul modello. La pagina nuova è `minibuffer`, che il piano non prevedeva: il prompt, la sua cronologia, `M-:`, l'anteprima di `C-x r t` e `minibuffer-read` non stavano in nessuna pagina, e in `completion` la SYNOPSIS avrebbe superato i cinquanta nomi. `regexp-opt` trova casa in `regexps`; `list-workers` passa al passo 8, con la pagina `background`, che oggi non ha una SYNOPSIS. Il registro `SETTINGS` documenta dieci impostazioni dell'area, con il loro valore all'avvio: `history-length`, `minibuffer-width`, `minibuffer-height`, `*minibuffer-read-function*`, `*completion-read-function*`, `*completion-filter-function*`, `*completion-invalidated-function*`, `*command-completion-function*`, `*command-arg-completion-function*` e `*key-capture-function*`. I loro nomi sono costanti di `editor/settings.rs`, che le primitive usano al posto delle copie che ciascuna teneva. `help-key` resta una `defvar` di `help.lisp`: il modulo la definisce e la usa, e il Rust la legge solo per l'aiuto a metà sequenza, che senza il modulo non c'è. Nei moduli `completion-window-height`, `completion-columns` e `manpage-path` passano da `setq` a `defvar` con docstring, e `help-history-length` da `defconst` a `defvar`. Le 42 docstring delle primitive dell'area e quelle dei sette moduli seguono il modello; le intestazioni dei moduli rimandano alle pagine, e i commenti di `common-keymaps.lisp` sul completamento, sui tasti, sulla ripetizione e sui lavori in background sono diventati rimandi.

Le pagine e le docstring dell'area dicevano cose false, ora corrette:

- nella striscia di completamento le frecce sinistra e destra muovono il cursore nel prompt, non la selezione: la muovono su e giù, e Tab sceglie;
- non tutti i candidati si cercano «ovunque nel nome»: i nomi dei file, e quelli dei buffer chiesti da un argomento, si completano dall'inizio;
- `C-M-i` non offre solo le parole dei buffer, ma anche i percorsi, le parole chiave del modo, i nomi del Lisp e quelli dei buffer, in ogni buffer;
- un tasto si cerca in quattro mappe, non in tre: tra quella transitoria e quella del modo c'è quella locale del buffer, che vim dà ai suoi stati;
- la domanda di `M-%` non trattiene i tasti che non conosce: è un'offerta, che qualunque altro tasto chiude; una domanda a un tasto come `y-or-n` accetta solo y e n, `C-g` compreso; e le mappe transitorie sono di tre tipi, con il filtro della striscia;
- `C-h a` cerca nei nomi, con un'espressione regolare, e non nelle descrizioni, come dicevano anche `first-steps` e `getting-help`;
- `K` in una pagina del manuale apre la pagina che ha il nome della parola sotto il cursore, non la descrizione di una funzione;
- la ricetta per spostare il tasto dell'aiuto non spostava `C-h m` e lasciava `C-h` un prefisso, quindi non restituiva Backspace a un terminale che manda `C-h`: ora ha le due righe che mancavano;
- `minibuffer-read` e `default-minibuffer-prompt` prendono un sesto argomento, la chiave della cronologia, che la firma taceva;
- `register-command` e `command-args` elencavano quattro codici di argomento su sette, e `read-key-sequence` taceva la mappa locale; `call-interactively` rimandava a `read-command-args`, che non esiste;

- gli esempi di `fuzzy-filter` e `regexp-opt` non si potevano incollare, per le virgolette precedute dalla barra, e quello di `define-key` usava `lisp-mode`, che non c'è; la docstring di `pending-command` aveva una fila di spazi a metà frase;
- l'intestazione di `completion.lisp` diceva che la striscia si scorre con n e p: si scorre con `C-n` e `C-p`.

Il passo ha trovato sei difetti del codice, che non corregge: li corregge la patch descritta qui sotto.

- `C-h b` elenca i tasti della mappa locale di un buffer, quelli degli stati di vim, sotto `Global -- everywhere`, come se valessero ovunque: `help--bindings-lines` conosce solo le sorgenti transient, mode e global;
- mentre una domanda è aperta, `key-binding` dà per un tasto che la domanda rifiuta ciò che il tasto farebbe senza di lei;
- `install-help-keys` non sposta `C-h m`, che `manpage.lisp` lega da sé, e lascia legati i tasti vecchi: la pagina ora dà la ricetta intera;
- nella striscia un candidato con una descrizione lunga riempie la sua colonna e si attacca al successivo, senza spazio in mezzo;
- `C-h f` su un comando mostra i codici grezzi dei suoi argomenti: «It asks for: fFind file:»;
- `M-:` lega `eval-expression-prompt`, che non è un comando: a `M-x eval-expression-prompt` l'editor risponde che il comando non esiste.

I difetti del passo 6 corretti. Una patch a parte corregge i sei difetti; `keymaps` non li descrive più tra i limiti, e ognuno ha un test che fallisce senza la correzione:

- `C-h b` elenca i tasti della mappa locale sotto un titolo loro, «Local» con il nome della mappa, tra quelli della domanda e quelli del modo;
- mentre una domanda è aperta, `key-binding`, `keymap-bindings` e `keys-with-prefix` danno solo i tasti della domanda, come la pressione di un tasto. La mappa locale continua a togliere `C-u` al lettore dell'argomento numerico, come prima: un test lo controlla;
- `define-key` con nil al posto del comando toglie il legame, e il prefisso rimasto senza tasti sotto di sé: prima non c'era modo di togliere un tasto, e nil lo legava a un comando che non fa nulla;
- `install-help-keys` lega sotto `help-key` anche `m`, il tasto del manuale, e toglie i tasti dell'aiuto dal tasto di prima, salvo quelli che l'utente ha legato ad altro; `manpage.lisp` lega `m` sotto `help-key` quando è già impostato, così un `help-key` scelto prima dei moduli non lega nulla sotto `C-h`. La ricetta di `getting-help` ha una riga in meno;
- nella striscia l'ultima colonna di ogni casella resta vuota, e un candidato lungo non si attacca più al successivo;
- `C-h f` su un comando dice gli argomenti a parole: «Its arguments: a file name (Find file:).»;
- `eval-expression-prompt` è un comando, e `M-x` lo raggiunge.

Passo 7, fatto. `modes`, `themes`, `overlays` e `virtual-text` sono riscritte sul modello. La pagina nuova è `writing-modes`, che il piano non prevedeva: come si scrive un modo — la colorazione con le regole e le regioni, la tabella della sintassi e le funzioni che la interrogano — stava solo nelle docstring e nei commenti di `rust-mode.lisp`, e in `modes` avrebbe mescolato chi usa i modi e chi li scrive. `modes` è la casa di `add-auto-mode`, degli hook e delle liste delle parole dei tre linguaggi; `themes` delle facce e dei colori. Le sezioni di `virtual-text` sul costo e sulle righe descrivevano l'interno ed escono dalla pagina: le dice già la documentazione di `ui/layout.rs`. In `theme.lisp` `current-theme` passa da `setq` a `defvar` con docstring, così ricaricare il modulo non dimentica il tema in vigore. Nessuna impostazione dell'area è letta dal Rust, e il registro `SETTINGS` non cambia. Le 30 docstring delle primitive dell'area e quelle dei quattro moduli seguono il modello; le intestazioni dei moduli rimandano alle pagine, il commento di `common-keymaps.lisp` sulla faccia della regione è diventato un rimando, e `themes/README.md`, che ripeteva la pagina, descrive i dieci temi classici e rimanda a `themes`.

Le pagine e le docstring dell'area dicevano cose false, ora corrette:

- `syntax-ppss` dà cinque elementi, non quattro;

- `face-style` non risponde nella forma che `set-face` prende: gli attributi seguono i colori invece di stare in una lista, e `themes` prometteva che un tema si potesse leggere così da un editor in funzione;
- `line-end-position`, che `virtual-text` e la docstring di `make-virtual-text` davano per la fine della riga, non esiste;
- gli esempi per il C di `editing` e di `indentation`, e quello della docstring di `electric-pair-enable`, usavano `cc-mode`, il nome del modulo, al posto di `cpp-mode`, il nome del modo, e non facevano nulla;
- gli esempi di `make-mode` e `add-auto-mode` usavano `lisp-mode`, che non c'è; quelli di `set-string-syntax`, `set-char-quote`, `syntax-ppss`, `balance-point` e `syntax-class` non si potevano incollare, per le virgolette precedute dalla barra;
- `set-syntax-pairs` non dichiara le coppie del modo: le aggiunge alle parentesi che ogni modo ha già, e in `risp-mode` le graffe restano coppie;
- `add-hook` prende il nome di una funzione e rifiuta una lambda; `post-command-hook` non gira dopo un comando che fallisce; la pagina non nominava `after-close-hook` né `after-set-visited-file-name-hook`;
- `modes` diceva ogni modo un file `.lisp`: `fundamental-mode`, `minibuffer-mode` e `isearch-mode` sono nel Rust;
- `overlays` diceva gli overlay le sole posizioni che l'editor tiene, e il testo virtuale ne tiene altre;
- l'intestazione di `risp-mode.lisp` chiamava comando `risp-refresh-vocabulary`, e la sua docstring rimandava a `eval-expression`, che non esiste; un commento di `rust-mode.lisp` nominava `set-mode-keywords`, che non esiste;
- l'esempio di `themes`, e quello di `configuration`, caricavano il modulo `theme`, che l'`init.lisp` predefinito carica già.

Due limiti noti sono ora scritti nelle pagine: `make-mode` su un modo che esiste lo rifà vuoto, chiavi e hook degli altri moduli compresi (SIN-4 nel rapporto), e una finestra flottante non mostra né la colorazione né gli overlay (TUI-6).

Il passo ha trovato sei difetti del codice, che non corregge: li corregge la patch descritta qui sotto.

- `reset-faces` rimette solo le quattordici facce che il Rust conosce; quelle che danno i moduli (`doc-comment`, `attribute`, `preprocessor`, `error`, `preview`, `results-here`, `results-target`, `manpage-bold`, `manpage-underline`) restano senza stile. Dopo `M-x reset-theme` le pagine del manuale perdono il grassetto e i segni di `M-g n` spariscono, e un tema che non le nomina le lascia spente;
- `monochrome` e `soft-contrast-gray` danno uno stile a `compilation-here` e `compilation-target`, che nessuno disegna, e non a `results-here`, `results-target`, `preview` e `preprocessor`; `themes/README.md` diceva che i temi così erano tre;
- i prompt di `set-face`, `set-kill-ring-max` e `set-undo-limit` cominciano con i due punti, «:Face: »: le specifiche sono scritte `"s:Face: "`, e `ArgSpec::parse` prende per prompt tutto ciò che segue il primo carattere;
- `risp-refresh-vocabulary` non è un comando, e ogni chiamata aggiunge le sue quattro regole senza togliere le vecchie: nove regole al caricamento, tredici dopo una chiamata, diciassette dopo due;
- `set-face` con gli attributi scritti come una stringa, `"bold"` invece di `'("bold")`, non accende nulla e non dice nulla, mentre un attributo sconosciuto è un errore;
- quando rifiuta una lambda, `add-hook` riferisce l'argomento sbagliato: `got "post-command-hook"` con il modo nil, `got "other"` con un modo.

I difetti del passo 7 corretti. Una patch a parte corregge i sei difetti; le pagine non li descrivono più tra i limiti, e ognuno ha un test che fallisce senza la correzione:

- la primitiva nuova `define-face` dà a una faccia lo stile con cui l'editor la distribuisce, che `reset-faces` rimette e che un tema che non la nomina lascia. I sette moduli che danno uno stile alle

loro facce lo fanno con `define-face` invece che con `set-face`, e dopo `M-x reset-theme` il manuale ha di nuovo il grassetto e `M-g n` i suoi segni;
- `monochrome` e `soft-contrast-gray` nominano `results-here` e `results-target` al posto dei nomi vecchi, e anche `preview` e `preprocessor`; un test controlla che ogni faccia nominata da un tema distribuito sia una faccia che l'editor disegna;
- le specifiche di `set-face`, `set-kill-ring-max` e `set-undo-limit` non hanno più i due punti, e un test li cerca in tutti i comandi;
- `add-syntax-rule` prende un quinto argomento, NAME: una regola aggiunta con il nome di una che il modo ha già ne prende il posto, dov'era. `risp-refresh-vocabulary` è un comando, e rifà le sue quattro regole invece di aggiungerne altre;
- `set-face` rifiuta gli attributi scritti come una stringa, e anche più di quattro argomenti;
- `add-hook` e `remove-hook` nominano l'argomento che rifiutano: la funzione quando è una lambda, il hook quando non è una stringa.

Passo 8, fatto. `writing-commands` e `background` sono riscritte sul modello, con la SYNOPSIS che non avevano, e la pagina nuova `shell` descrive `M-!`, `C-c !`, il buffer dell'uscita con le sue righe e i suoi tasti, come si ferma un comando, le quattro primitive di `shell.rs` e i limiti. `writing-commands` è la casa di `defcommand`, `string-prefix-p` e `string-suffix-p`, e dà i sette codici degli argomenti; `background` lo è di `background-call`, `define-worker`, `stop-worker`, `running-workers`, `list-workers`, `emit-event`, `work-list`, `work-option`, `set-work-option`, `work-options` e `worker-fuel`, e descrive `fiber` e `yield`, forme speciali che `C-h f` non trova, con i punti in cui un `yield` è ammesso, provati forma per forma. Da `background` sono usciti la storia di `M-x compile` che teneva il thread condiviso, le sette regole scritte per chi sviluppa e la sezione su `completion-invalidate`, che ha casa in `completion` e ora vi è solo citata; la SEE ALSO nomina solo pagine. `guide`, `rsedit` e `compile` nominano `shell`.

Le docstring delle tredici primitive di `workers.rs` e `shell.rs` seguono il modello, e le cinque che superavano il limite sono scese: `background-call` da 3 145 caratteri a 756, nella forma definitiva dell'esempio delle regole, `define-worker` da 1 944 a 700, `shell-command-start` da 1 795 a 620, `parse-overstrike` da 1 075 a 460 e `shell-command-to-string` da 1 042 a 467. Seguono il modello anche quelle di `shell.lisp`, di `defcommand`, di `string-prefix-p` e `string-suffix-p` e di `list-workers`; il ragionamento di `string-suffix-p` e di `shell--show` è passato in commenti `;;`, e le intestazioni di `shell.lisp` e `commands.lisp` rimandano alle pagine. `worker-fuel` entra nel registro `SETTINGS`, con la docstring e il valore 200 000 all'avvio: l'editor lo leggeva senza che avesse un valore, e `C-h v worker-fuel` non lo trovava. Il nome è una costante di `editor/settings.rs`, che `workers.rs` usa. `shell-output-window-height` è una `defvar`: con `defconst` un `setq` scritto nell'`init.lisp` prima del modulo andava perso quando il modulo si caricava. Il commento dell'`init.lisp` predefinito su `worker-fuel` rimanda alla pagina. L'elenco dei nomi senza casa perde il blocco del passo 8, ventuno nomi, quello delle docstring lunghe i suoi cinque, e `worker-fuel` esce dai nomi citati sconosciuti; `defun`, `defvar` e `setq`, che le pagine ora citano, si aggiungono alle forme speciali, e i nomi che nessuno cita più escono dall'elenco.

Le pagine e le docstring dell'area dicevano cose false, ora corrette:

- `writing-commands` elencava `(buffer-name)`, che non esiste: il nome del buffer corrente lo dà `(current-buffer)`;
- `writing-commands` dava sei codici degli argomenti, e la docstring di `defcommand` quattro: sono sette, `s`, `n`, `b`, `f`, `p`, `P` e `r`;
- l'esempio di `insert-date` inseriva la data seguita dall'a capo che `date` stampa;
- `writing-commands` diceva che una funzione ridefinita con `M-:` vale subito, e che `C-h e` mostra il backtrace di ciò che fallisce: quello che `M-:` definisce non resta (il primo difetto qui sotto), e il log riporta il messaggio, mentre il backtrace si apre solo con `debug-on-error`. `configuration` e `diagnostics`, che propongono `M-:` per provare una riga e per caricare un modulo, ora dicono che le definizioni non restano;

- l'esempio della docstring di `shell-command-to-string` non si poteva incollare, per le virgolette precedute dalla barra, e quella di `parse-overstrike` raccontava la propria storia.

Le pagine scrivono i limiti che il passo ha provato sull'editor, in un terminale: un comando che chiede una password al terminale viene fermato dal sistema e aspetta `C-c C-k`; un demone avviato dal comando non si ferma con lui, e se tiene l'uscita il buffer non finisce finché non la lascia. Su Windows un comando si ferma subito, e solo la shell: questo viene dal codice, e non è provato. `background` dice anche una regola del Lisp che un lavoro in background incontra subito: un `setq` di un nome senza valore, dentro una funzione, crea una variabile di quella funzione, e il risultato che il lavoro vi lascia sparisce con lei.

Il passo ha trovato quattro difetti del codice, che non corregge:

- quello che `M-:` definisce non resta. `eval-expression-confirm`, in `minibuffer.lisp`, valuta il testo con `eval-string-safe` dentro la propria funzione, e una `defun`, una `defvar`, una `defcommand`, un `setq` di un nome nuovo e ciò che definisce un file caricato con `(eval-file ...)` finiscono nel suo ambiente locale e spariscono al ritorno; anche una funzione che esiste già, ridefinita, resta quella di prima. Una `defcommand` lascia a `M-x` un comando che non può eseguire (`UndefinedFunction`). Un tasto legato a `'(eval-file "mine")` invece carica il file nell'ambiente globale, e `writing-commands` lo propone;
- con il codice `n`, una risposta che non è un numero, o nessuna risposta, arriva al comando come 0: `convert_arg`, in `primitives/commands.rs`, legge il testo con `unwrap_or(0.0)`, e il prompt non chiede di nuovo;
- ogni `M-!` apre una striscia nuova sotto quelle che ci sono, invece di mostrare l'uscita in quella del comando di prima come fa `compile` con la sua: in un terminale di trenta righe il terzo comando lascia una riga sola alla finestra in cui si lavorava;
- `shell-command-start` è registrata come comando: `M-x` la propone accanto a `shell-command`, e la versione di `M-x` avvia il comando senza mostrare nulla, perché il buffer dell'uscita nasce senza finestra.

I difetti del passo 8 corretti. Una patch a parte corregge i quattro difetti; le pagine non li descrivono più tra i limiti, e ognuno ha un test che fallisce senza la correzione:

- `M-:` valuta l'espressione come `init.lisp`, fuori da ogni funzione, con la primitiva nuova `eval-string-top-level`, che ha casa in `minibuffer`; `eval-file` valuta il file allo stesso modo da dovunque la si chiami, e cerca il file nel `lisp-path` del punto della chiamata. Una `defun`, una `defvar` o una `defcommand` scritte con `M-:`, e ciò che definisce un file caricato con `(eval-file ...)`, restano. `writing-commands` torna a proporre `M-:` per provare un comando, e `configuration` e `diagnostics` tornano come prima del passo 8;
- con il codice `n`, una risposta che non è un numero, o nessuna risposta, è rifiutata: l'area dei messaggi dice «Please enter a number» e il prompt chiede di nuovo, come in Emacs;
- `M-!` mostra l'uscita nella striscia del comando di prima, come `compile` riusa la sua; una striscia chiusa con `q`, o data a un altro buffer, non viene ripresa, e il comando dopo ne apre una sua;
- `shell-command-start` è una funzione e non più un comando.

Passo 9, fatto. `vim` è riscritta sul modello. La SYNOPSIS passa da tre nomi a ventuno: i sette comandi, `vim-default-buffer-p` e le tredici variabili che si impostano o si leggono, e la pagina è la casa di tutti. Le tabelle dei tasti restano, scritte per chi preme i tasti e non per chi ha scritto il modulo, e due sezioni sono nuove: WHAT VIM REMEMBERS, le otto variabili che vim condivide fra i buffer e che un programma può leggere, e KEYMAPS, le tre mappe in cui `define-key` cambia quello che fa un tasto, con un esempio. KEYS IN VIM INSERT MAP elenca i cinque tasti di Insert legati a un comando, che i test controllano; `C-u`, legato a una forma, è descritto nel testo. I tasti di Normal restano nelle tabelle, perché la mappa lega ogni tasto stampabile a una forma che lo passa a `vim--key`, e una sezione KEYS direbbe solo questo. EX COMMANDS aggiunge `:w!`, `:noh`, `:h` e il `\t` delle sostituzioni, che il modulo accettava e la pagina non diceva. Escono i paragrafi per chi sviluppa, sulle primitive generiche che il modulo usa e sul «globalized buffer-local mode»

di Emacs, e il consiglio sull'ordine in cui caricare il modulo, che `configuration` dà per tutti i moduli; la SEE ALSO, che mancava, nomina nove pagine, e `guide` riporta il NAME nuovo.

Le 76 funzioni e le 15 variabili di `vim.lisp` hanno una docstring sul modello. Le 68 funzioni private che non l'avevano ne hanno una di una riga o due, che dice che cosa fanno o che cosa restituiscono; quelle dei comandi, di `vim-default-buffer-p` e delle variabili sono riscritte per chi le legge con `C-h f` e `C-h v`, e la più lunga, quella di `global-vim-mode`, ha 270 caratteri. L'intestazione di `vim.lisp` dice che cosa fa il modulo e rimanda alla pagina; gli altri commenti spiegano il codice, e restano alla fase C. L'elenco dei nomi senza casa perde i diciotto nomi del passo 9, e quello delle funzioni senza docstring i suoi 68.

Con il passo 9 la fase B è finita. Gli elenchi delle eccezioni che il passo 1 aveva aperto sono vuoti: ogni nome pubblico ha una casa, ogni funzione dei moduli Lisp ha una docstring, nessuna docstring supera i 1 000 caratteri e ogni nome citato è un nome che l'editor definisce. Resta pieno solo l'elenco dei nomi citati che a ragione non sono dell'editor: le forme speciali come `defun` e `yield`, i hook, i simboli che una funzione prende o restituisce. Un nome pubblico nuovo senza casa, una funzione nuova senza docstring o una docstring troppo lunga fanno fallire i test.

La pagina vecchia era esatta quasi ovunque. Diceva che `I` e `A` in una selezione a blocco non ci sono, e invece inseriscono sulla riga del punto, come in Normal; e che i comandi prendono il prefisso grezzo di Emacs, mentre `C-u -` accende vim invece di spegnerlo (il quarto difetto qui sotto).

La pagina scrive i limiti che il passo ha provato sull'editor, tasto per tasto: `C-o` in Insert apre una riga, come senza vim; un mark è una posizione, e il testo inserito prima se ne allontana; in un blocco `I` e `A` inseriscono sulla riga del punto, e un cambiamento si ripete su ogni riga solo quando il suo testo sta su una riga; il conteggio davanti a `R` non conta, e `.` non ripete una sostituzione; `=` senza il modulo `indent` non fa nulla; `C-w h`, `j`, `k` e `l` vanno alla finestra dopo, e `z` seguito da un tasto qualsiasi ricentra la finestra; i tasti che sfogliano contano l'altezza del frame. In Visual `:` lascia la selezione, e `'<,'>` va scritto.

Il passo ha trovato quattro difetti del codice, che non corregge:

- `ge` dal primo carattere di una parola va alla fine della parola prima di quella precedente, e `dge` cancella altrettanto di più: dalla `t` di `three`, in `one two three`, va alla `e` di `one`. Il ramo `"ge"` di `vim--motion` va all'inizio della parola con `vim--word-backward`, e da lì alla fine della parola prima; ma dal primo carattere `vim--word-backward` va già all'inizio della parola precedente;
- un conteggio davanti a `@` o `@@` conta anche per il primo comando della macro: con `a` che cancella un carattere, `3@a` ne cancella cinque. `kmacro-call-keys` ripete i tasti subito, prima che `vim--reset` azzeri il conteggio, e il primo `x` lo trova ancora;
- `ap` è `ip`, e le righe vuote dopo il paragrafo restano: il ramo `"p"` di `vim--object` prende la fine da `forward-paragraph`, che si ferma prima delle righe vuote, e con `a` non le aggiunge;
- `C-u -` davanti a `M-x vim-mode` o `M-x global-vim-mode` accende invece di spegnere: il prefisso grezzo è il simbolo `-`, che i due comandi trattano come un valore vero; `C-u -1` spegne. Il test `interactive_global_toggle_honors_negative_prefix_argument` non se ne accorge: preme `C--`, che non è legato a nulla e quindi non dà un prefisso, e passa perché il tasto senza prefisso spegne quello che è acceso.

I difetti del passo 9 corretti. Una patch a parte corregge i quattro difetti, e uno in più trovato correggendoli; la pagina non li descrive più tra i limiti, e ognuno ha un test che fallisce senza la correzione:

- `ge` va alla fine della parola prima da ogni punto di una parola, dal primo carattere come dal mezzo; anche da uno spazio, da dove andava una parola più indietro, e dopo un segno di punteggiatura, che saltava. `dge` cancella fino a lì;
- un conteggio davanti a `@` o `@@` dice solo quante volte la macro si ripete: `vim--reset` lo azzera prima che la macro parta;
- `ip` e `ap` seguono le regole di Vim, in una funzione nuova, `vim--paragraphs`, invece di `backward-paragraph` e `forward-paragraph`. `ap` prende anche le righe vuote dopo il paragrafo, o

quelle prima quando dopo non ce ne sono; su righe vuote `ip` prende quelle, e `ap` quelle e il paragrafo dopo; una riga fatta solo di spazi bianchi è vuota, e un conteggio conta un gruppo di righe vuote come un paragrafo. Così `ip` smette di sbagliare in tre casi: su una riga vuota cancellava il paragrafo prima, con un conteggio contava i paragrafi e non i gruppi di righe vuote, e su un paragrafo in fondo al buffer ne lasciava l'ultima riga;

- `C-u -` davanti a `M-x vim-mode` o `M-x global-vim-mode` spegne, come `C-u -1`. Il difetto aveva due metà. I due comandi leggevano il simbolo `-` come un valore vero, e una funzione nuova, `vim--turn-on-p`, lo legge come meno uno. E `M-x` non passava il prefisso a nessun comando: lo azzerava quando il suo tasto finiva, prima di leggere il nome, e `C-u 3 M-x forward-char` avanzava di un carattere. Ora `M-x` lo consegna al comando che lancia, come in Emacs, con un secondo argomento facoltativo di `execute-extended-command`, PREFIX, e `completion` lo dice. Il test `interactive_global_toggle_honors_negative_prefix_argument` ora dà davvero `C-u -`, da un buffer in cui vim è spento, perché dove è acceso `C-u` è uno dei suoi tasti; e lo dà due volte, così che un interruttore non basti a farlo passare. La pagina `vim` dice come vim legge un prefisso;
- il difetto in più: un conteggio davanti a `gg` o `ge` restava al comando dopo, e `4gg` seguito da `dd` cancellava quattro righe. Nel ramo di `g` di `vim--pending-key` un `if` aveva due forme nel ramo else, e il Lisp dell'editor valuta solo la prima: `(vim--reset)` non veniva mai eseguito. Era l'unico `if` così nei moduli e nei temi.

Il Lisp dell'editor ignora in silenzio le forme del ramo else dopo la prima, dove Emacs le valuta tutte: è una differenza dell'interprete, che il passo 10 porta nel rapporto tra le schede di `risp`.

Passo	Area	Pagine	Primitive (`primitives/`)	Moduli Lisp e pagine nuove
2	avvio, configurazione, diagnostica (fatto)	`rsedit`, `guide`, `first-steps`, `configuration`	da `general.rs`: `eval-file`, `log`, `all-logs`, `backtrace`	`debug`, `commands`; nuova: `diagnostics` (il log nella cache, `*Messages*`, `C-h e`, il backtrace, le righe del watchdog)
3	scrivere e modificare (fatto)	`editing`, `rectangles`, `macros`	`edits`, `verbs`, `region`, `rectangle`, `macros`, `comments`	`indent`, `electric-pair`, `clipboard`; nuove: `indentation`, `killing`
4	file, buffer, finestre (fatto)	`files`, `buffers`, `windows`	`io`, `buffers`, `windows`, `ui`, `mouse`, `ask`	`dired`, `buffer-list`; nuove: `dired`, `mouse`; le impostazioni `auto-save` e `watch-files`
5	cercare e navigare (fatto)	`searching`, `compile`	`isearch`, `replace`, `results`, `scan`, `match_data`	`occur`, `compile`; nuova: `regexps`
6	prompt, completamento, tasti, aiuto (fatto)	`completion`, `keymaps`, `getting-help`	`minibuffer`, `completion`, `help`, `commands`, il resto di `general`	`minibuffer`, `completion`, `completion-at-point`, `help`, `manpage`, `preview`, `common-keymaps`; nuova: `minibuffer`
7	modi e aspetto (fatto)	`modes`, `themes`, `overlays`, `virtual-text`	`modes`, `theme`, `overlays`, `virtual_text`	`rust-mode`, `risp-mode`, `cc-mode`, `theme`; nuova: `writing-modes`

Passo	Area	Pagine	Primitive (`primitives/`)	Moduli Lisp e pagine nuove
8	estendere l'editor (fatto)	`writing-commands` `background`	`workers`, `shell`	`shell`; nuova: `shell`
9	vim (fatto)	`vim`	—	`vim`

4.3 Fase C — il codice e i commenti (passi 10–21)

Il passo 10 aggiorna il rapporto sui problemi: ogni scheda verificata sul codice di quel momento, il campo `stato`, le schede dell'interprete separate per il repository `risp`. Va prima del codice perché la classificazione dei commenti (§ 2.13) deve sapere quali problemi sono risolti.

Gli altri passi seguono l'ordine delle dipendenze di `lib.rs`, dal basso. Ogni passo comincia con una proposta breve degli spostamenti — file da dividere, unire o rinominare — che approvi prima della patch. Poi la patch: il rustdoc del sottosistema secondo le regole, i commenti classificati, il capitolo del manuale con le sue decisioni e i suoi diagrammi, i commenti dei test del sottosistema (decisione 8). Il comportamento non cambia: tutti i test passano senza modifiche, salvo gli spostamenti che seguono quelli del codice.

Passo	Parte	Righe	Note
10	`doc/rapporto-problemi.typ`	1 244	80 schede da verificare
11	`work/`	2 935	assorbe `workmanager.typ`
12	`paths.rs`, `buffer/`	3 075	—
13	`text/`, `input.rs`, `commands.rs`	2 047	—
14	`managers/`	2 936	la regola delle coppie con `editor/`
15	`editor/`	4 838	`boot.rs`, `keys.rs`, `frame.rs`
16	`primitives/`, modifica del testo: `edits`, `verbs`, `region`, `rectangle`, `macros`, `comments`, `match_data`, `scan`	4 353	docstring già fatte nella fase B
17	`primitives/`, file, finestre e lavori: `io`, `buffers`, `windows`, `ui`, `mouse`, `ask`, `shell`, `workers`, `lisp_job`, `args`	4 995	`io.rs` è il file più lungo: 1 818 righe
18	`primitives/`, il resto: `commands`, `completion`, `general`, `help`, `isearch`, `minibuffer`, `modes`, `overlays`, `replace`, `results`, `theme`, `virtual_text`, `mod`	5 953	—
19	`modes/`, `feature/`	2 896	—
20	`ui/`, `src/` (la TUI)	3 408	—
21	`core/lisp/`: intestazioni e commenti	6 422	docstring già fatte nella fase B

I criteri con cui si propongono gli spostamenti:

- **La mappa di `lib.rs` è quella vera.** Oggi elenca un modulo `lisp/` che è nel crate `risp`. Un test la confronta con le dichiarazioni `mod`.
- **Un concetto per file, e il file ha il nome del concetto.** Un file oltre le 800 righe circa deve avere un motivo: oggi `primitives/io.rs` (1 818), `primitives/edits.rs` (1 567), `ui/windows.rs` (1 437), `work/manager.rs` (1 094), `text/search.rs` (952), `primitives/results.rs` (936), `src/tui.rs` (924).

- **Una regola sola per le coppie `editor/X` e `managers/X`**: ce ne sono otto (`buffers`, `commands`, `kill_yank`, `log`, `macros`, `modes`, `runtime`, `windows`). La regola si scrive in `lib.rs` e nel manuale, e vale per tutte.
- **La stessa parola per la stessa cosa**: lavoro, worker e task, per esempio.
- **In un file, prima ciò che è pubblico**, poi gli aiuti privati, nell'ordine in cui il lettore li incontra.
- **Un file di test per argomento**, con il nome del modulo che prova.

4.4 Fase D — il manuale (passo 22)

- `architettura-editor.typ` diventa il manuale per sviluppatori: l'introduzione, i diagrammi d'insieme (strati, thread, ordine dei lock), i capitoli scritti nella fase C, le ricette, il glossario.
- `workmanager.typ` si toglie, assorbito nel passo 11.
- L'appendice generata delle primitive si toglie: è una copia del registro, e `C-h a` la sostituisce.
- Da questo documento escono l'inventario e il piano; restano le regole.
- I PDF si rigenerano.

4.5 Come si consegna ogni passo

Come le patch 1–9. La patch si prepara su una copia di lavoro, si prova con `git am` su una copia pulita, con l'intera suite di test, e confrontando gli avvisi di clippy e di `cargo doc` con quelli della base. Arriva con il PDF dei documenti toccati e con la spiegazione, file per file, di che cosa cambia e di dove differisce da questo piano.

5 Le decisioni

Le risposte alle domande del passo 0.

1. **Il README.** `man/README.md` si riscrive con il modello delle pagine. Il README del progetto diventa una descrizione essenziale secondo le pratiche di GitHub, un portfolio sobrio dell'applicazione, con l'elenco dei documenti e dei loro destinatari: `README.md` al posto di `README.org`.
2. **La SYNOPSIS come casa dei nomi**, con il rimando automatico in `C-h f`: sì.
3. **Un solo elenco dei moduli**: `build.rs` copia tutta la directory.
4. **I PDF in git**: restano, rigenerati nello stesso commit della sorgente; i due orfani si tolgono.
5. **`lisp-fibers.typ`**: lo toglie l'autore.
6. **`workmanager.typ`**: ciò che è ancora valido confluisce nel manuale nel passo 11, e il documento si toglie nella fase D.
7. **La storia nei commit**: le patch si applicano con `git am`.
8. **I commenti dei test**: si ripuliscono con le stesse regole insieme al loro sottosistema, nella fase C.
9. **Il manuale scritto durante la fase C**: sì.
10. **Le correzioni dell'inventario**: quelle di codice e test nel passo 1, quella di `configuration.txt` nel passo 2.